Introduction to Computer Science
Spring 2026
hgl235@nyu.edu

Homework 7

Professor HG Locklear

Imperial Particle

AP BP CP

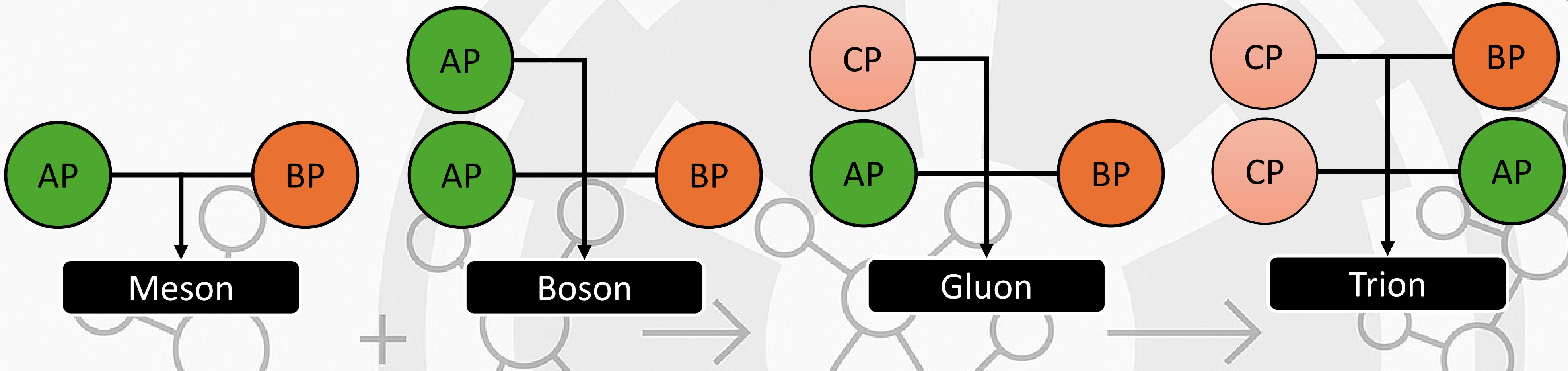
Particle	Symbol	Mass	Charge	Spin	Power
Alpha	AP	5×10^{-2}	Positive	Left	1.5×10^{6}
Beta	BP	10×10^{-2}	Negative	Right	2.5×10^{6}
Gamma	CP	15×10^{-2}	Positive	Right	3×10^{6}

Each ImperialParticle is an elementary particle that has a symbol, mass, charge, spin, and power.

MONOMER

POLYMER

Imperial Cluster

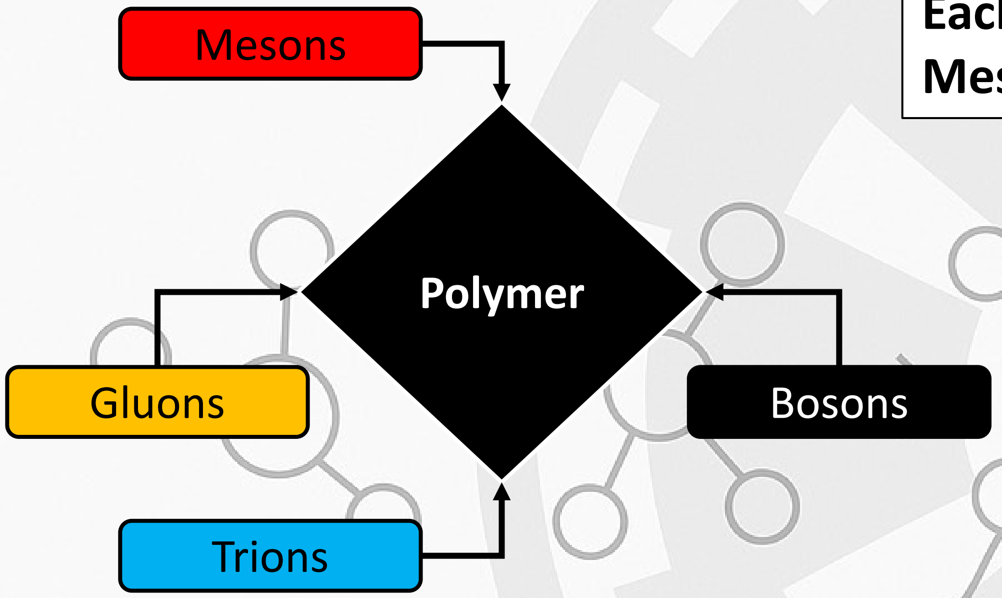


Cluster	Alpha	Beta	Gamma
Meson	1	1	0
Boson	2	1	0
Gluon	1	1	1
Trion	1	1	2

Each ImperialCluster is composed of a specified number of AlphaParticles, BetaParticles, and GammaParticles.

POLYMER

Imperial Polymer



Each ImperialPolymer is composed of a specified number of Meson, Bosons, Gluons, and Trions.

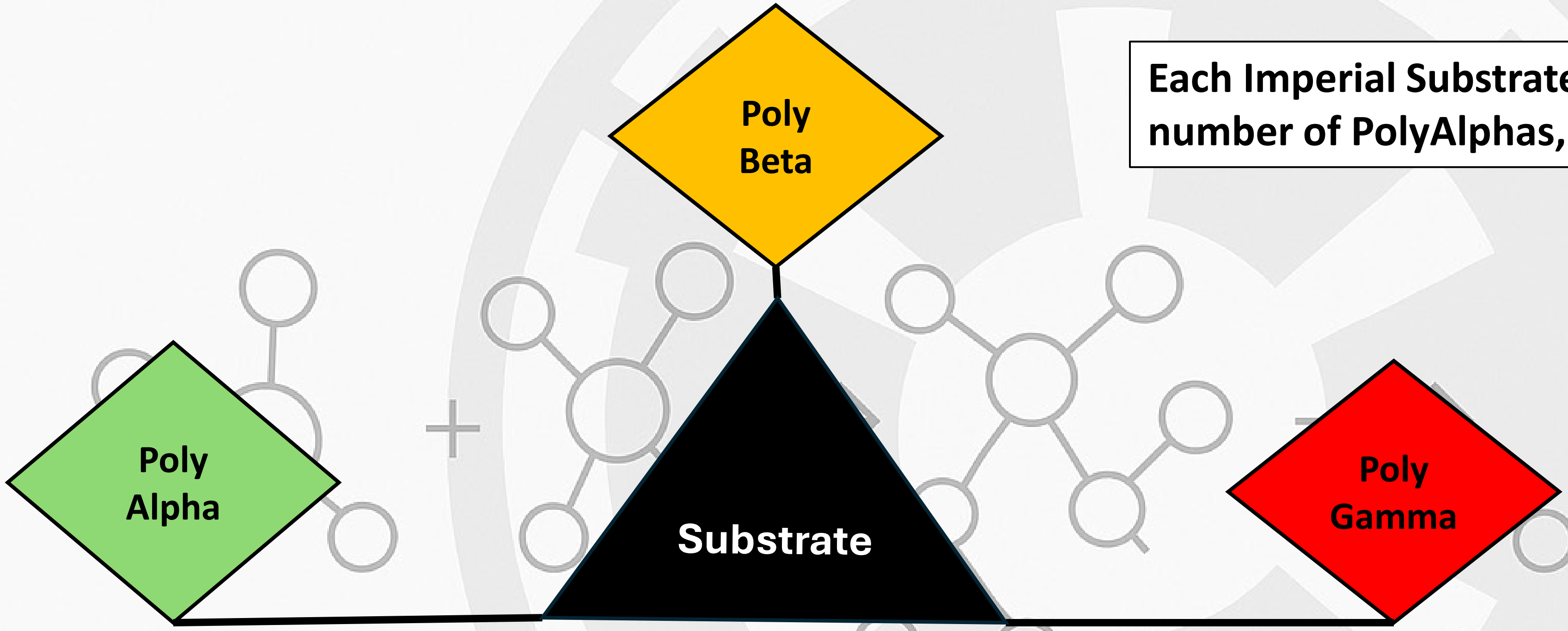
MONOMER

Polymer	Mesons	Bosons	Gluons	Trions
PolyAlpha	1	1	2	1
PolyBeta	2	2	1	1
PolyGamma	1	1	1	1

POLYMER

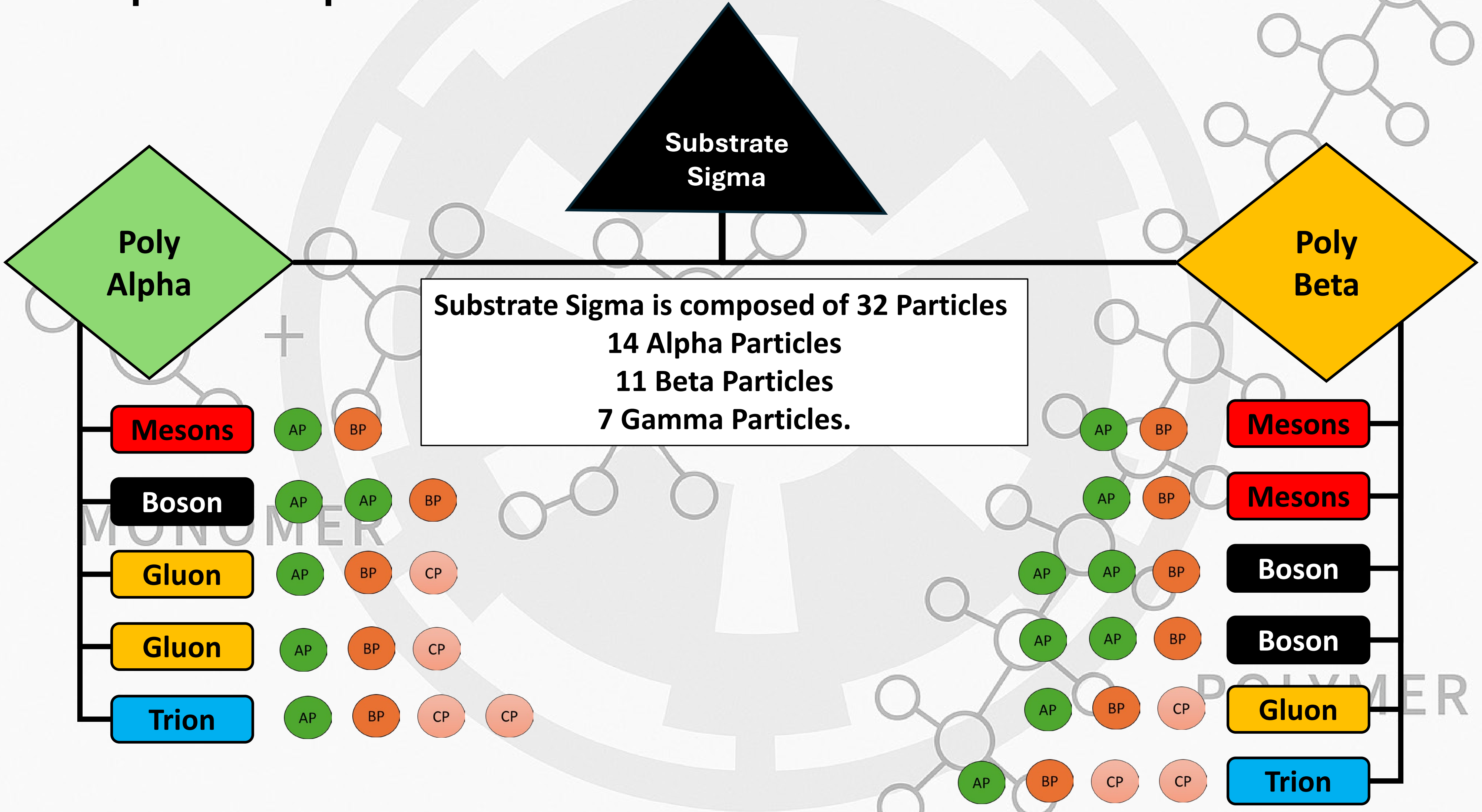
Imperial Substrate

Each Imperial Substrate is composed of a specified number of PolyAlphas, PolyBetas, and PolyGammas



Substrate	Symbol	PolyAlpha	PolyBeta	PolyGamma
Substrate_Sigma	SS	1	1	0
Substrate_Tau	TS	0	2	1
Substrate_Zeta	ZS	1	0	1

Example: Imperial Substrate



Program Structure

- HW7_Locklear
  - JRE System Library [JavaSE-23]
  - src
    - lockklear.SUBSTRATE
      - ImperialSubstrate.java
      - Substrate_Sigma.java
      - Substrate_Tau.java
      - Substrate_Zeta.java
    - locklear.CLUSTER
      - Boson.java
      - Gluon.java
      - ImperialCluster.java
      - Meson.java
      - Trion.java
    - locklear.ENUMS
      - ParticleCharge.java
      - ParticleSpin.java
    - locklear.INTERFACES
      - Elemental.java
    - locklear.MAIN
      - Gene.java
    - locklear.PARTICLE
      - Alpha.java
      - Beta.java
      - Gamma.java
      - ImperialParticle.java
    - locklear.POLYMER
      - ImperialPolymer.java
      - PolyAlpha.java
      - PolyBeta.java
      - PolyGamma.java

Create your Program Structure and main method EXACTLY as shown replace my name with yours where appropriate

```java
public class Gene {

    public static void main(String[] args) {
        TreeMap<Integer,ImperialSubstrate> subs = Elemental.buildSubstrates(10, 10, 10);
        Elemental.breakDownSubstrate(subs.get(23));
    }
}
```

ImperialParticle

<Abstract>
ImperialParticle

-String pID
-String pType
-String symbol
-double mass
-ParticleCharge charge
-ParticleSpin spin
-double power
+static int pCount = 1

ImperialParticle(String pType, String symbol, double mass, ParticleCharge charge, ParticleSpin spin, double power)

+toString(): **String**
+abstract displayParticle(): **void**

The ImperialParticle constructor increments **pCount** by 1 at each call

The **pID** (particle id) is formatted as :
P-[particle type]**-**[pCount]

Alpha

Alpha( )

+displayParticle(): **void**

Beta

Beta()

+displayParticle(): **void**

Gamma

Gamma()

+displayParticle(): **void**

ImperialCluster

TreeMap<String,ArrayList<ImperialParticle>>

<Abstract>
ImperialCluster

-String cID
-**String** cType
-int *clusterCount* = 0

ImperialCluster( )

+displayAllParticles( ): **void**
+abstract displayCluster(): **void**
+abstract displayTotalMassPower(): **void**

The ImperialCluster constructor increments **clusterCount** by 1 at each call

The **cID** (cluster id) is formatted as : **C-**[cluster type]**-**[clusterCount]

MONOMER

Meson

+displayCluster(): **void**
+displayTotalMassPower(): **void**

Boson

+displayCluster(): **void**
+displayTotalMassPower(): **void**

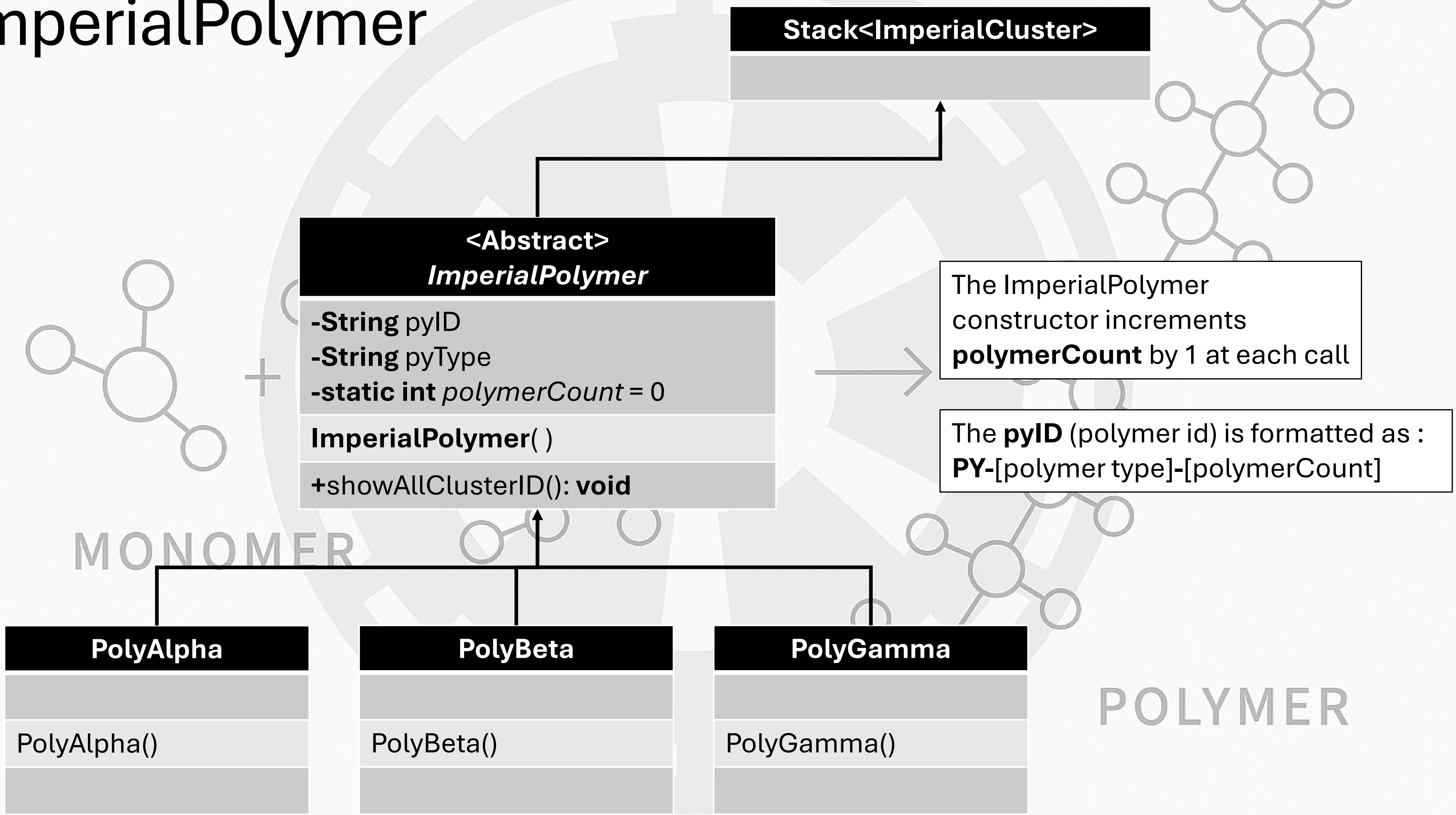
Gluon

+displayCluster(): **void**
+displayTotalMassPower(): **void**

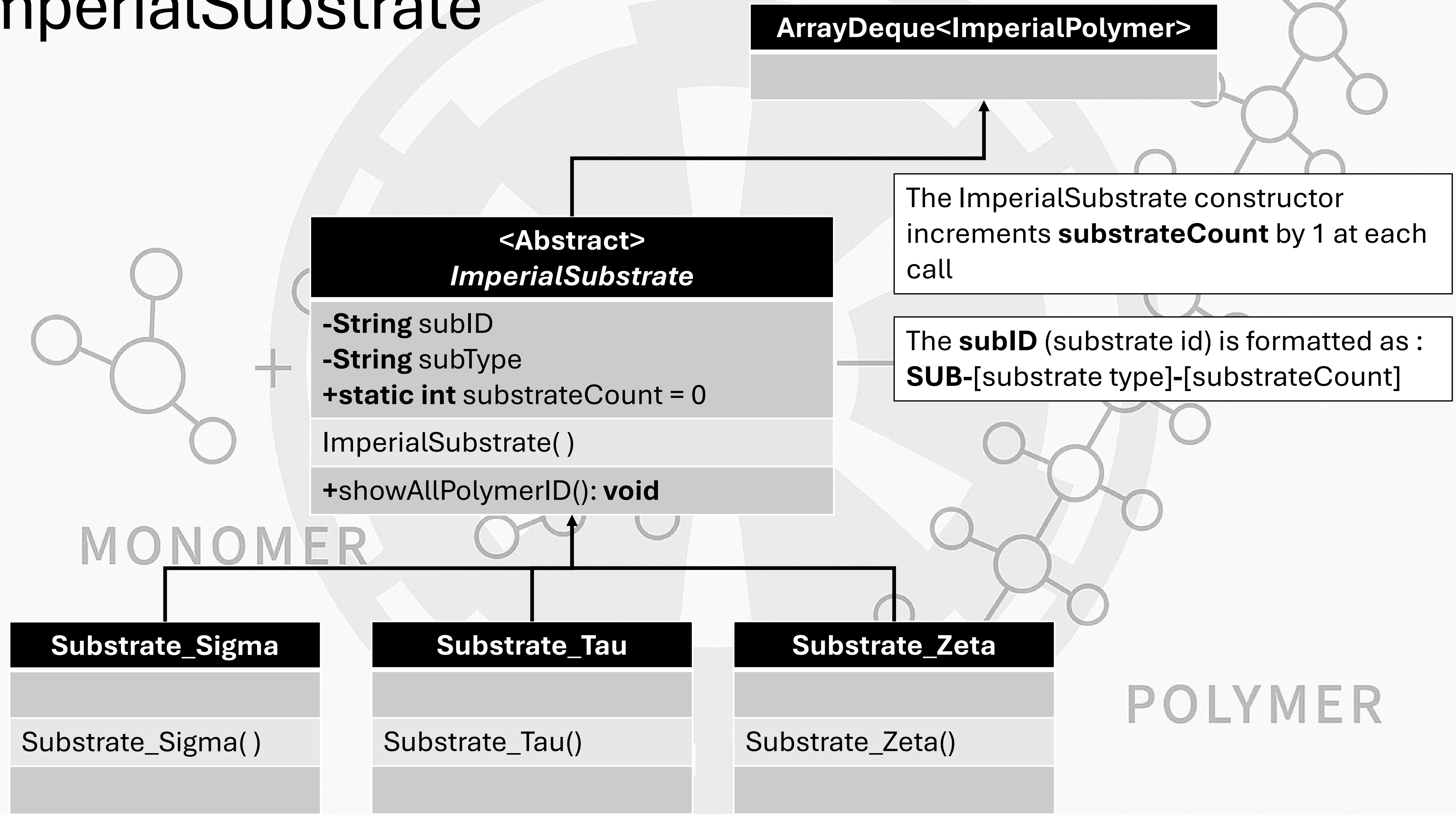
Trion

+displayCluster(): **void**
+displayTotalMassPower(): **void**

ImperialPolymer



ImperialSubstrate



Elemental Interface

<<Elemental>>
+static getParticleIDs(ArrayList<ImperialParticle> A): **String** **+static** buildSubstrates(int sigma, int tau, int zeta): **TreeMap<Integer, ImperialSubstrate>** **+static** breakDownSubstrate(ImperialSubstrate sub): **void**

Method	Input	Processing	Output
getParticleIDs	A	Returns all the pIDs of the ImpericalParticle objects in **A** as a single String	String
buildSubstrates	sigma tau zeta	Creates the specified number and type of ImperialSubstrate objects specified by **sigma**, **tau**, and **zeta** and stores them in a TreeMap where their build number (order in which they were built) is the key	TreeMap
breakDownSubstrate	sub	Prints to the console a representation of the ImperialSubstrate object **sub** as shown in the **Expected Output**	Console

Method Specification Chart

Class	Method	Input	Processing	Output
ImperialParticle	toString	None	Returns a String representation of the ImperialParticle object in any format	String
ImpericalParticle (subclasses)	displayParticle	None	Prints to the console a String formatted as [Particle Type]: [pID]	Console
ImperialCluster	displayAllParticles	None	Prints to the console the **pID** of each particle in the ImperialCluster object	Console
ImperialCluster (subclasses)	displayCluster	None	Calls the **displayParticle** method for each ImperialParticle in the ImperialCluster object	Console
	displayTotalMassPower	None	Prints to the console the total mass and power of the ImperialCluster object (accumulated total of all particles)	Console
ImperialPolymer	showAllClusterID	None	Prints to the console the **cID** of all clusters in the ImperialPolymer object	Console
ImperialSubstrate	showAllPolymerID	None	Prints to the console the **pyID** of all polymers in the ImperialSubstrate object	Console

Expected Output

Problems Javadoc Declaration Console

```
<terminated> Gene (38) [Java Application] C:\Users\msgth\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot
Substrate_Zeta
**Polymers**
PY-PolyAlpha-55 [PolyAlpha]
        C-Messon-289
        C-Boson-290
        C-Gluon-291
        C-Gluon-292
        C-Trion-293
PY-PolyGamma-56 [PolyGamma]
        C-Messon-294
        C-Boson-295
        C-Gluon-296
        C-Trion-297
```